

3.1 TF 坐标变换与数学基础

所属阶段： 第三阶段 · 坐标变换与机器人建模

讲次数量： 11 节

预计时长： ~130 分钟

前置依赖： 1.5 RViz 2 可视化工具入门；2.1–2.7 ROS 2 核心通信机制（节点、Topic、Service、Action、Launch、Parameter）

学习目标

完成本节后，学员将能够：

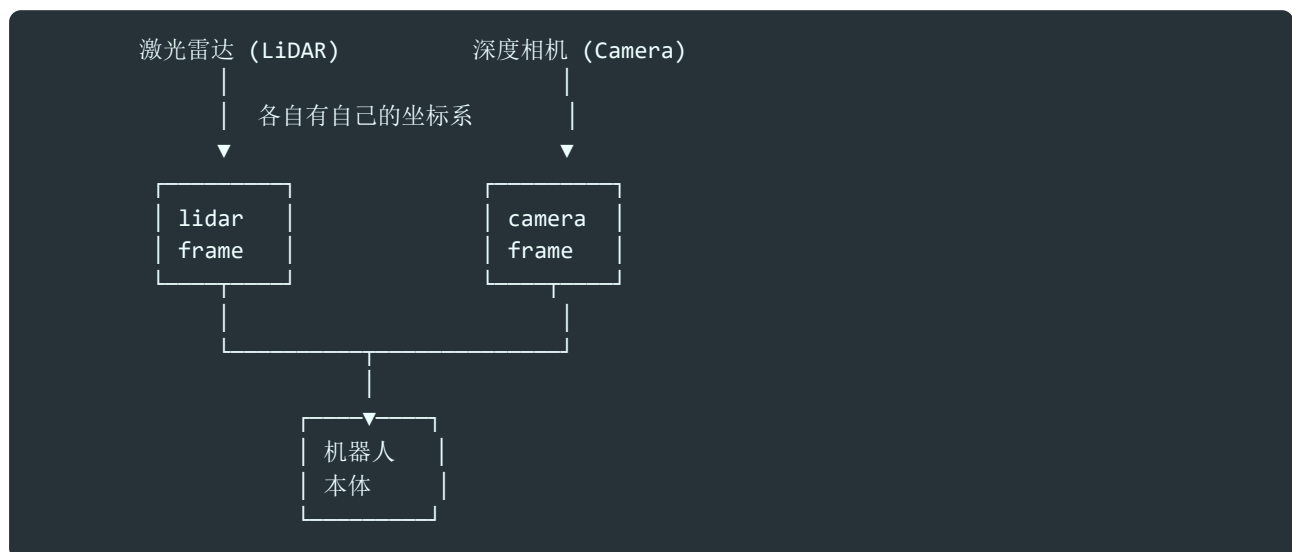
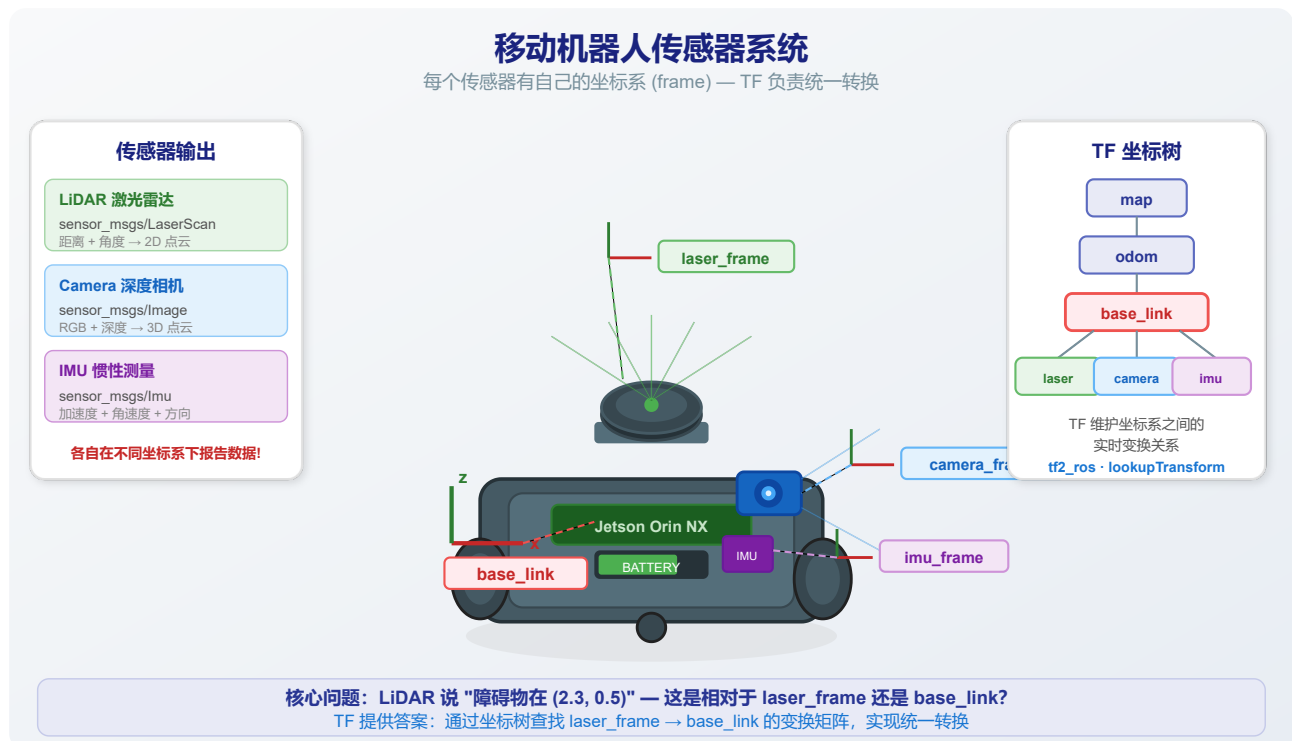
- 解释多传感器融合中坐标系不一致带来的问题，以及 TF 系统的解决思路
- 区分 TF（运行时变换）与 URDF（静态模型描述）的职责，理解 TF 树的树形结构
- 说明 Robot State Publisher 如何读取 URDF 并发布关节变换
- 在 RViz 2 中可视化 TF 树，使用 `tf2_tools` 和命令行工具调试变换
- 掌握移动机器人标准坐标系（map / odom / base_link / sensor frames）及其定位约定
- 用平移向量、旋转矩阵和齐次变换矩阵表示机器人位姿
- 手工计算坐标系间的变换，理解 TF 底层数学原理

讲次 1: TF 解决了什么问题——多传感器融合中的坐标系挑战

知识点

真实机器人的“坐标系混乱”

想象一辆配备激光雷达、深度相机和 IMU 的移动机器人：



每个传感器都在**自己的坐标系**下输出数据： - 激光雷达报告：“障碍物在 (2.3, 0.5, 0) 米处” - 相机报告：“目标在图像像素 (320, 240)，深度 1.2 米” - IMU 报告：“绕 Z 轴角速度 0.05 rad/s”

感知节点想把激光点云和相机检测结果**融合**时，面临一个根本问题：**这些坐标是在不同参考系下表达的，不能直接相加或比较。**

没有 TF 时代价高昂

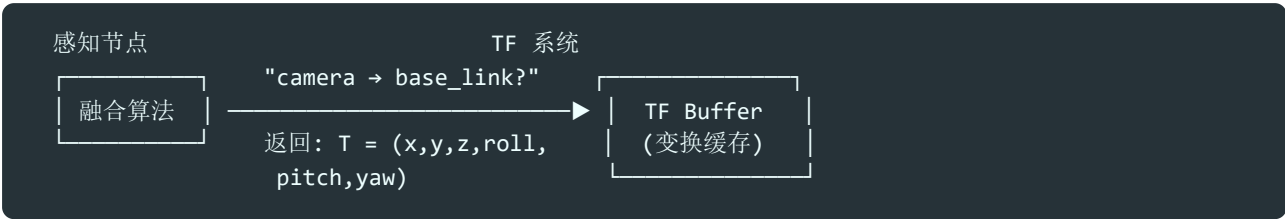
在 TF 出现之前，开发者需要：

问题	后果
在每个节点中硬编码传感器安装位置	换传感器或调整安装角度需改多处代码
手动维护坐标变换公式	容易出错，难以验证
各模块使用不同坐标约定	集成测试频繁失败，调试周期极长
无法动态更新变换（如机械臂关节运动）	只适用于完全静态的安装关系

TF 的核心价值

TF (Transform) 是 ROS 提供的分布式坐标变换管理系统。它解决的核心问题是：

在任何时刻，查询任意两个坐标系之间的空间关系（平移 + 旋转）。



TF 将"坐标系关系"从业务代码中解耦出来，变成系统级基础设施。

典型应用场景

场景	TF 的作用
多传感器融合	将激光点云从 <code>laser_frame</code> 变换到 <code>base_link</code> 统一处理
导航	将里程计位姿从 <code>odom</code> 变换到 <code>map</code> 全局坐标
机械臂	查询末端执行器 <code>tool0</code> 相对于 <code>base_link</code> 的位姿
可视化	RViz 2 将所有传感器数据显示在同一世界坐标系下
运动规划	规划器在 <code>map</code> 坐标系下生成路径，控制器在 <code>base_link</code> 下执行

ROS 2 中的 TF 技术栈

ROS 2 Jazzy 中，TF 相关核心包：

包名	作用
tf2	核心数学库（变换计算、四元数、矩阵）
tf2_ros	ROS 2 接口（Buffer、Listener、Broadcaster）
tf2_geometry_msgs	与 geometry_msgs 的互操作
tf2_tools	命令行工具（view_frames 等）
robot_state_publisher	根据 URDF 发布关节变换

消息类型：

```
# 查看 TF 相关消息定义
ros2 interface show tf2_msgs/msg/TFMessage
ros2 interface show geometry_msgs/msg/TransformStamped
```

TransformStamped 是一条变换记录的核心结构：

```
std_msgs/Header header      # 时间戳 + 父坐标系名（frame_id）
string child_frame_id       # 子坐标系名
geometry_msgs/Transform transform
  Vector3 translation        # 平移（x, y, z）
  Quaternion rotation        # 旋转（四元数 x, y, z, w）
```

动手体验：感受坐标系问题

启动 TurtleBot3 fake node，它会同时发布机器人模型、关节状态和 TF：

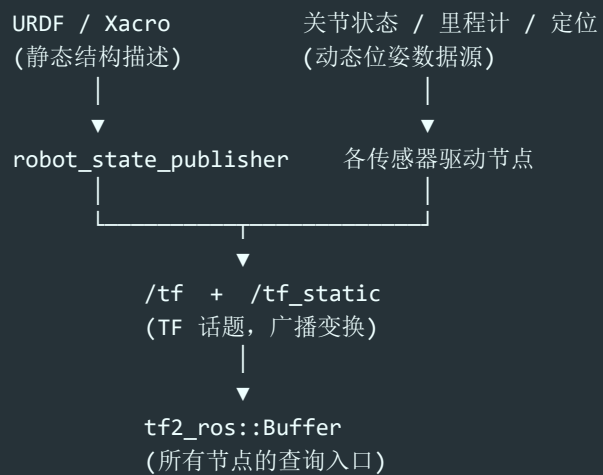
```
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py
```

另开终端，查看 TF 数据流：

```
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep tf
ros2 topic echo /tf_static --once
```

你应该能看到 /tf 和 /tf_static。其中 /tf_static 包含 base_link -> base_scan、base_link -> imu_link 等固定安装关系。

小结: TF 在 ROS 生态中的位置



关键点

- 多传感器各自拥有独立坐标系，融合前必须统一到同一参考系
 - TF 是 ROS 的分布式坐标变换管理系统，提供"任意两坐标系间关系"的查询能力
 - TF 将坐标变换从业务代码解耦，是导航、感知、可视化的基础设施
 - ROS 2 Jazzy 中 TF 核心为 `tf2` + `tf2_ros`，消息载体为 `/tf` 和 `/tf_static` 话题
 - `TransformStamped` 包含父坐标系、子坐标系、平移和旋转（四元数）四项关键信息
-

讲次 2：TF 与 URDF 的关系、TF 树的概念

知识点

URDF 与 TF：模型与运行时

很多初学者混淆 URDF 和 TF。它们密切相关，但职责不同：

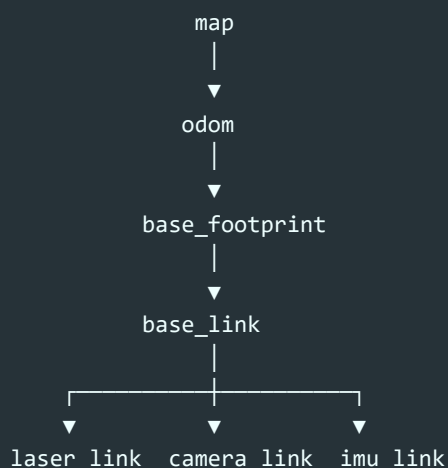
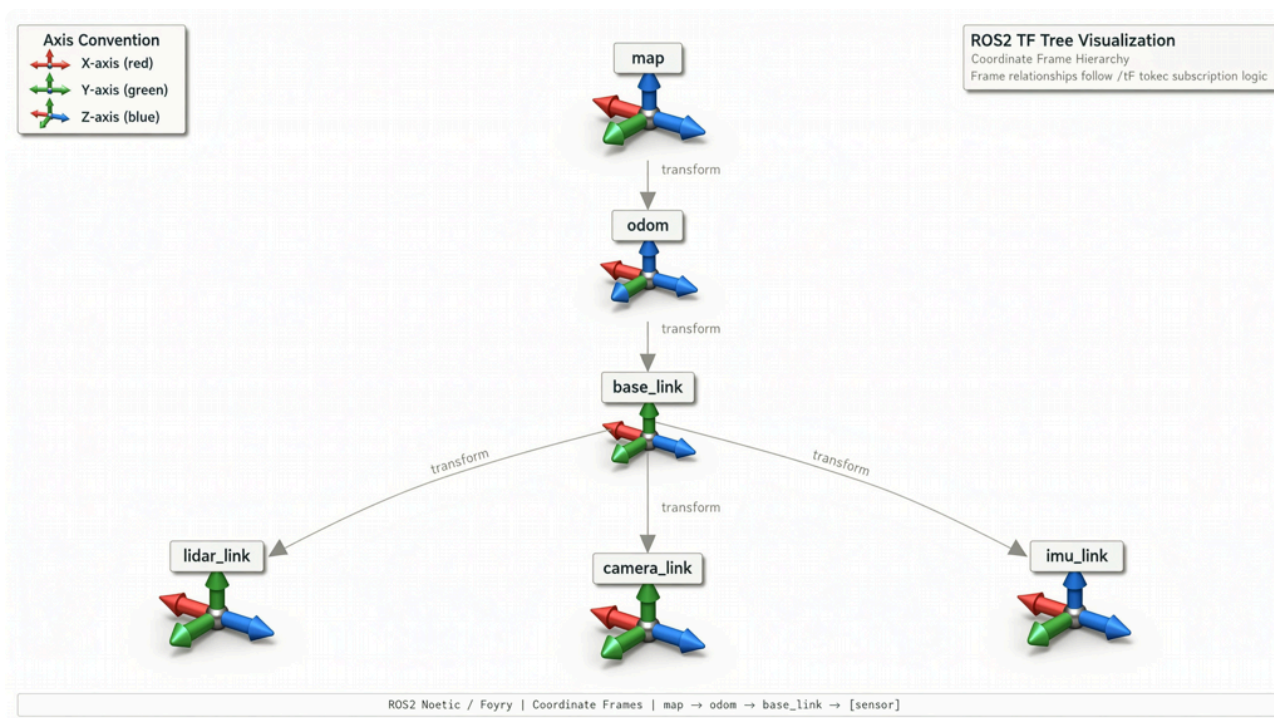
维度	URDF	TF
本质	机器人结构的静态描述文件 (XML)	坐标变换的运行时广播系统
内容	Link (连杆)、Joint (关节) 的几何与运动关系	任意时刻两坐标系间的平移+旋转
更新频率	仅在修改模型时变化	高频动态更新 (可达 kHz)
存储位置	文件系统 (<code>.urdf</code> / <code>.xacro</code>)	内存中的 TF Buffer
典型发布者	<code>robot_state_publisher</code>	<code>robot_state_publisher</code> 、里程计、定位、传感器驱动

可以这样理解：

URDF 回答“机器人由哪些部件和关节组成”；TF 回答“在已有变换链和指定时间下，两个 frame 之间的相对位姿”。TF 本身不是定位算法，也不保证系统一定存在 `map` 或 `world frame`。

TF 树 (TF Tree)

一条可查询的机器人坐标链应组成有向树 (Directed Tree)：



核心规则：

1. **唯一父节点**：每个坐标系 (child frame) 只能有一个父坐标系 (parent frame)
2. **无环**：不允许出现 $A \rightarrow B \rightarrow C \rightarrow A$ 的循环引用
3. **查询连通性**：需要互相变换的 frame 必须位于同一棵连通树中

TF2 Buffer 可以暂时包含多个互不连通的子树，例如某个传感器驱动先于机器人模型启动；此时跨子树查询会失败，但不是“系统中出现多个根就立即非法”。自变换、空 child frame、同一 child 被多个来源竞争发布等情况会被忽略、警告或造成不稳定结果。

为什么必须是树，不能是图？

如果允许“多父节点”，同一坐标系可能通过不同路径得到**不一致的变换结果**，系统无法判断哪个是正确的。树结构保证了**任意两节点间有且仅有一条路径**，变换结果唯一确定。

合法（树）：

map → odom → base_link

非法（环）：

base_link → camera



TF 树的命令行观察

安装 TurtleBot3 相关包后（课程后续项目会用到）：

```
sudo apt install -y \
  ros-jazzy-turtlebot3-description \
  ros-jazzy-turtlebot3-fake-node \
  ros-jazzy-tf2-tools
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
```

启动 TurtleBot3 fake node 发布 TF 树：

```
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py
```

生成 TF 树 PDF 可视化：

```
# 新终端
source /opt/ros/jazzy/setup.bash
ros2 run tf2_tools view_frames
# Jazzy 会生成带时间戳的 frames_YYYY-MM-DD_HH.MM.SS.pdf 和 .gv
ls -la frames_*.pdf frames_*.gv
evince frames_*.pdf # 或用浏览器打开
```

`view_frames` 输出内容示例：

```
base_link:
  parent: 'base_footprint'
  broadcaster: 'default_authority'
  rate: 10000.0
  most_recent_transform: 0.0
  oldest_transform: 0.0
  buffer_length: 0.0
base_scan:
  parent: 'base_link'
  broadcaster: 'default_authority'
...
```

URDF 如何映射到 TF 树

URDF 中每个 `<link>` 对应 TF 树中的一个 **frame**（坐标系）。`<joint>` 定义了父子 link 之间的变换：


```
<!-- URDF 片段 -->
<joint name="laser_joint" type="fixed">
  <parent link="base_link"/>
  <child link="laser_link"/>
  <origin xyz="0.1 0 0.2" rpy="0 0 0"/>
</joint>
```

映射关系：

URDF	TF 树
<code><link name="base_link"></code>	→ frame: base_link
<code><link name="laser_link"></code>	→ frame: laser_link
<code><joint> parent→child</code>	→ 变换: base_link → laser_link
<code><origin xyz rpy></code>	→ 平移 + 旋转值

/tf 与 /tf_static 的区别

话题	发布内容	典型来源	特点
/tf	随时间变化的变换	关节状态、里程计、定位	有时间戳，可被插值/外推
/tf_static	固定不变的变换	URDF 中的 fixed joint	无时间衰减，一次发布持续有效

```
# 查看 TF 话题
ros2 topic info /tf --verbose
ros2 topic info /tf_static --verbose

# 查看实时 TF 数据
ros2 topic echo /tf --once
ros2 topic echo /tf_static --once
```

坐标系命名惯例

命名模式	含义	示例
*_link	物理连杆坐标系	base_link , arm_link
*_frame	传感器或虚拟坐标系	laser_frame , map_frame
base_footprint	机器人在地面的投影	2D 导航常用
odom , map	里程计 / 全局地图坐标系	导航栈标准

命名规则建议：全小写，下划线分隔，语义清晰。

关键点

- URDF 是静态结构描述，TF 是运行时变换系统；二者通过 `robot_state_publisher` 桥接

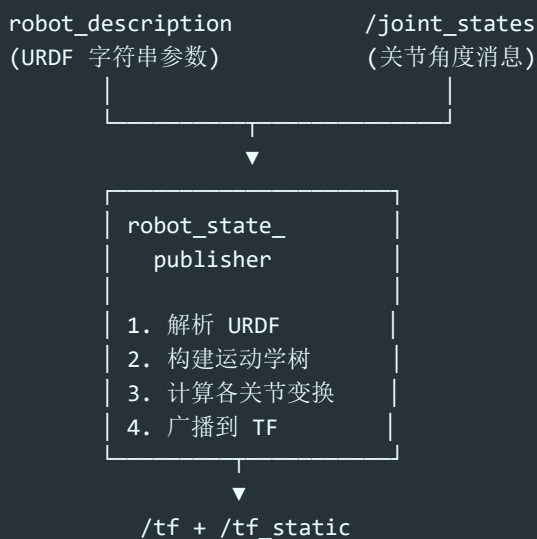
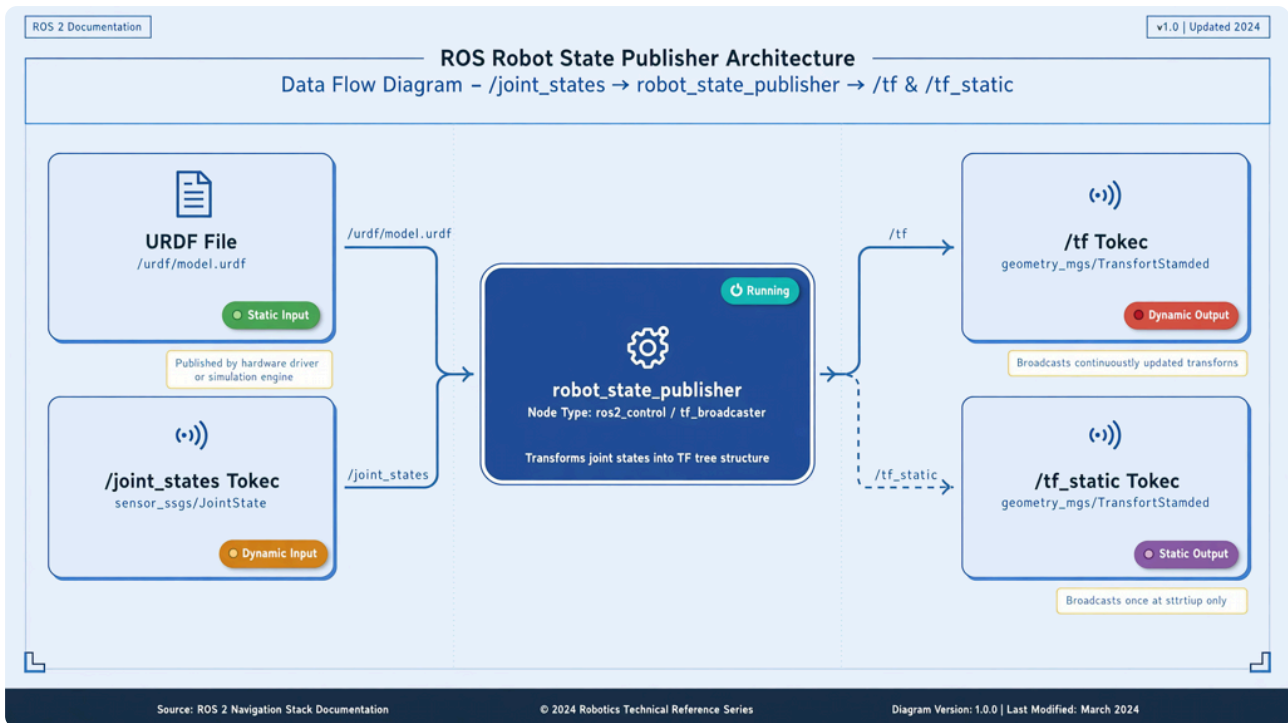
- 一条可用的 TF 变换链应保持树形：child frame 只有一个父 frame 且无环；Buffer 中可暂时存在多个不连通子树
 - 树结构保证任意两坐标系间变换路径唯一，结果确定
 - `/tf` 发布动态变换，`/tf_static` 发布固定关节变换
 - URDF 的每个 link 对应 TF 树的一个 frame，joint 定义父子变换关系
-

讲次 3: Robot State Publisher 如何与 URDF 协作

知识点

Robot State Publisher 的角色

`robot_state_publisher` 是 ROS 2 中连接 **URDF 模型** 与 **TF 树** 的核心节点。它的工作流程：



输入一: robot_description 参数

URDF 内容以字符串形式存入 `robot_description` 参数:

```
# 查看参数
ros2 param get /robot_state_publisher robot_description
```

Launch 文件中通常这样加载：

```
# Python Launch 示例（概念预览，3.2 节将详细实践）
from launch import LaunchDescription
from launch_ros.parameter_descriptions import ParameterValue
from launch.substitutions import Command
from launch_ros.actions import Node

def generate_launch_description():
    urdf_content = Command(['cat ', 'path/to/robot.urdf'])
    return LaunchDescription([
        Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            parameters=[{
                'robot_description': ParameterValue(urdf_content, value_type=str)
            }]
        )
    ])
])
```

输入二：/joint_states 话题

对于**运动关节**（revolute、prismatic 等），`robot_state_publisher` 需要实时关节角度才能计算变换：

```
ros2 interface show sensor_msgs/msg/JointState
```

```
string[] name      # 关节名称列表
float64[] position # 关节位置（弧度或米）
float64[] velocity # 关节速度（可选）
float64[] effort   # 关节力矩（可选）
```

消息示例：

```
header:
  stamp: {sec: 0, nanosec: 0}
  frame_id: ''
name: ['wheel_left_joint', 'wheel_right_joint']
position: [1.57, 1.57]
velocity: []
effort: []
```

对于**固定关节**（fixed），变换在启动时计算一次，发布到 `/tf_static`，无需 `/joint_states`。

内部计算逻辑

固定关节 (fixed):

变换 = URDF 中 `<origin xyz="..." rpy="..."/>`
→ 发布到 `/tf_static`, 永不改变

运动关节 (revolute / prismatic / continuous):

变换 = URDF 中 `<origin>` × 关节运动模型 (如绕 Z 轴旋转 θ)
→ 每次收到 `/joint_states` 更新, 发布到 `/tf`

以旋转关节为例:

URDF origin joint position (θ)

↓ ↓

$T_{parent \rightarrow child} = T_{origin} \times Rot_z(\theta)$

手动启动 Robot State Publisher

```
# 终端 1: 准备 URDF 文件 (使用系统自带的 TurtleBot3 Gazebo URDF)
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
export URDF_FILE=/opt/ros/jazzy/share/turtlebot3_gazebo/urdf/turtlebot3_burger.urdf
```

```
# 终端 1: 启动 robot_state_publisher
ros2 run robot_state_publisher robot_state_publisher \
  --ros-args -p robot_description:="$(cat $URDF_FILE)"
```

```
# 终端 2: 发布关节状态 (模拟关节角度)
source /opt/ros/jazzy/setup.bash
ros2 topic pub /joint_states sensor_msgs/msg/JointState \
  "{name: ['wheel_left_joint', 'wheel_right_joint'], position: [0.0, 0.0]}"
```

```
# 终端 3: 验证 TF 输出
source /opt/ros/jazzy/setup.bash
ros2 topic echo /tf_static --once
ros2 run tf2_ros tf2_echo base_link base_scan
```

tf2_echo 输出示例:

```
At time 0.0
- Translation: [-0.032, 0.000, 0.172]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
           in RPY (radian) [0.000, -0.000, 0.000]
```

joint_state_publisher 与 joint_state_publisher_gui

当没有真实硬件时, 用这两个包模拟关节状态:

包	作用
<code>joint_state_publisher</code>	发布全零或默认关节位置
<code>joint_state_publisher_gui</code>	提供 GUI 滑块，手动调整关节角度

```
# 配合 robot_state_publisher 使用
ros2 run joint_state_publisher_gui joint_state_publisher_gui
```

GUI 中拖动滑块时，`/joint_states` 实时更新，TF 树中对应 link 随之运动。

常见问题排查

现象	可能原因	解决方法
TF 树断裂	URDF 有孤立 link	检查所有 link 是否通过 joint 连通
关节不动	无 <code>/joint_states</code> 发布	启动 <code>joint_state_publisher</code>
关节名不匹配	JointState 的 name 与 URDF 不一致	核对名称拼写
机器人位置偏移	<code><origin></code> 值错误	检查 URDF 中 joint origin
启动报错	URDF 语法错误	<code>check_urdf robot.urdf</code> 验证

验证 URDF 语法：

```
sudo apt install liburdfdom-tools
check_urdf /path/to/robot.urdf
urdf_to_graphviz /path/to/robot.urdf # 生成结构图
```

关键点

- `robot_state_publisher` 读取 `robot_description` (URDF) 和 `/joint_states`，计算并广播 TF
 - 固定关节变换发布到 `/tf_static`；运动关节变换发布到 `/tf`
 - `/joint_states` 消息中的 `name` 必须与 URDF 关节名严格匹配
 - 仿真/调试时用 `joint_state_publisher_gui` 手动驱动关节
 - `check_urdf` 和 `tf2_echo` 是排查 TF 问题的首选工具
-

讲次 4: 【LAB】在 RViz 2 中可视化机器人 TF

知识点

实验目标

本实验将完成以下任务： 1. 启动 TurtleBot3 的 URDF 模型和 TF 发布 2. 在 RViz 2 中配置 TF、RobotModel 显示 3. 使用 TurtleBot3 fake node 发布 `/joint_states` 和里程计 TF 4. 用 `tf2_echo` 和 `view_frames` 验证变换关系

环境准备

```
# 安装依赖（若尚未安装）
sudo apt update
sudo apt install -y \
  ros-jazzy-turtlebot3-description \
  ros-jazzy-turtlebot3-fake-node \
  ros-jazzy-turtlebot3-gazebo \
  ros-jazzy-tf2-tools \
  ros-jazzy-robot-state-publisher \
  ros-jazzy-rviz2

# 配置环境变量
echo "export TURTLEBOT3_MODEL=burger" >> ~/.bashrc
source ~/.bashrc
source /opt/ros/jazzy/setup.bash
```

本节配套源码

本节源码位于仓库：

```
stage3_code/tf_math_demo
```

建议单独放到临时工作空间中构建：

```
mkdir -p ~/ros2_ai_stage3_ws/src
cp -r /path/to/ros2_ai/stage3_code/tf_math_demo ~/ros2_ai_stage3_ws/src/
cd ~/ros2_ai_stage3_ws
source /opt/ros/jazzy/setup.bash
colcon build --packages-select tf_math_demo
source install/setup.bash
```

纯数学验证：

```
ros2 run tf_math_demo transform_math_demo
```

TF 点变换验证（需先启动 TurtleBot3 fake node）：

```
ros2 run tf_math_demo tf_point_transform_demo
```

步骤 1：启动 TF 发布链

打开终端 1——启动 TurtleBot3 fake node:

```
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py
```

该 launch 会启动 `turtlebot3_fake_node`、`robot_state_publisher` 和 RViz 2，并发布 `/joint_states`、`/tf`、`/tf_static` 与 `/robot_description`。

验证 TF 正在发布:

```
# 终端 2
source /opt/ros/jazzy/setup.bash
ros2 topic list | grep tf
ros2 topic echo /tf_static --once
ros2 topic hz /joint_states
```

步骤 2：启动 RViz 2 并配置显示

如果你使用课程配套源码，可以直接加载已验证的 RViz 配置:

```
source /opt/ros/jazzy/setup.bash
source ~/ros2_ai_stage3_ws/install/setup.bash
ros2 run rviz2 rviz2 -d "$(ros2 pkg prefix tf_math_demo)/share/tf_math_demo/rviz/turtlebot3_tf.rviz"
```

如果手动从空白 RViz 配置开始，按以下步骤配置:

(1) 设置 Fixed Frame

Global Options → Fixed Frame → `base_link`

Fixed Frame 是 RViz 的全局参考坐标系，所有显示项都变换到此系下渲染。

(2) 添加 RobotModel

左下角 Add → By display type → RobotModel → OK

属性	值
Description Source	Topic
Description Topic	<code>/robot_description</code>

若机器人模型未出现，检查 `robot_state_publisher` 是否正常运行。

(3) 添加 TF 显示

Add → TF → OK

TF 显示配置：

属性	建议值	说明
Show Names	✓	显示坐标系名称
Show Axes	✓	显示 RGB 坐标轴
Show Arrows	✓	显示父子连接箭头
Marker Scale	0.3	坐标轴大小
Frame Timeout	15	超时秒数

RGB 轴颜色约定：

X 轴 = 红色 (Red) → 前方
Y 轴 = 绿色 (Green) → 左侧
Z 轴 = 蓝色 (Blue) → 上方

(4) 添加 Grid (可选)

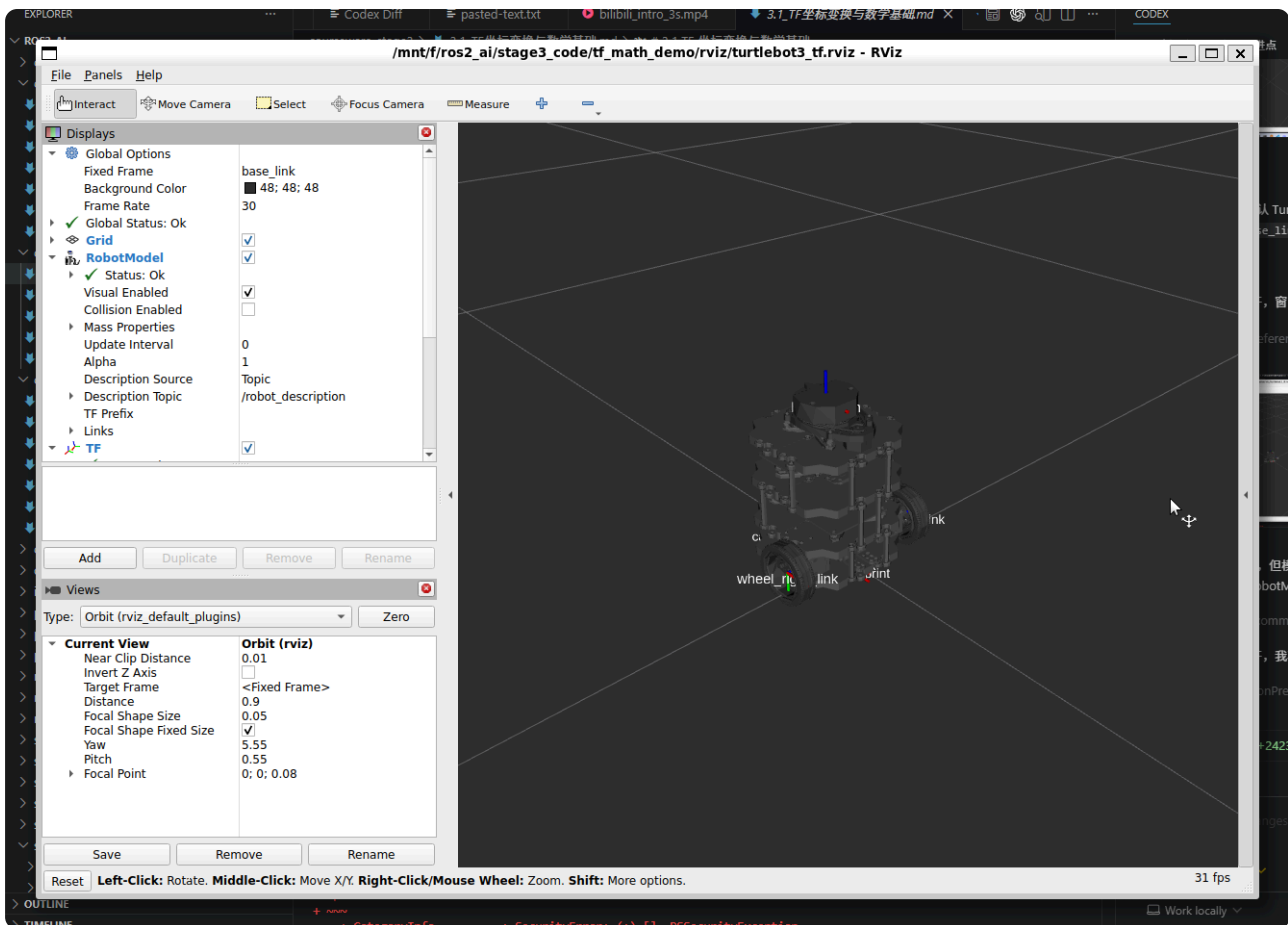
Add → Grid → OK

步骤 3：观察 TF 树结构

在 RViz 2 的 TF 面板中，展开树形列表：

```
base_footprint
└─ base_link
    ├── base_scan      (激光雷达)
    ├── caster_back_link (后万向轮)
    ├── imu_link       (IMU)
    ├── wheel_left_link (左轮)
    └── wheel_right_link (右轮)
```

Burger 模型没有 `camera_link`。后续如果切换到 Waffle Pi 或自定义相机模型，再观察相机相关 frame。



步骤 4: 命令行验证变换

```
# 查询 base_link 到 laser 的变换
ros2 run tf2_ros tf2_echo base_link base_scan

# 查询 base_link 到 IMU 的变换
ros2 run tf2_ros tf2_echo base_link imu_link
```

记录变换值，与 URDF 中对应 joint 的 `<origin>` 对比验证。

生成 TF 树 PDF:

```
ros2 run tf2_tools view_frames
ls -la frames_*.pdf frames_*.gv
```

步骤 5: 保存 RViz 配置

```
File → Save Config As → ~/ros2_ws/turtlebot3_tf.rviz
```

后续可通过 `-d` 参数加载:

```
ros2 run rviz2 rviz2 -d ~/ros2_ws/turtlebot3_tf.rviz
```

实验练习

完成以下任务以巩固理解：

练习	要求
练习 1	将 Fixed Frame 改为 <code>base_scan</code> ，观察模型显示如何变化
练习 2	在 TF 面板中取消勾选 <code>caster_back_link</code> ，观察可视化差异
练习 3	用 <code>tf2_echo</code> 查询 <code>wheel_left_link</code> → <code>base_link</code> 的变换
练习 4	发布 <code>/cmd_vel</code> ，观察 fake node 产生的 <code>odom</code> → <code>base_footprint</code> 动态 TF

常见实验问题

问题	解决方案
RViz 中机器人不显示	检查 Fixed Frame 是否设为 <code>base_link</code> ；确认 <code>/robot_description</code> 有数据
TF 显示 "No transform"	等待几秒让 TF buffer 填充；检查 <code>robot_state_publisher</code> 是否运行
轮子不转	确认 <code>turtlebot3_fake_node</code> 已启动；检查 <code>/joint_states</code> 频率
坐标轴过大/过小	调整 TF → Marker Scale 参数

关键点

- RViz 2 中 Fixed Frame 决定全局渲染参考系，TF 实验通常设为 `base_link`
- RobotModel 从 `/robot_description` 获取 URDF，TF 显示展示坐标系树
- RGB 轴约定：X 红（前）、Y 绿（左）、Z 蓝（上）—— ROS 标准右手坐标系
- `turtlebot3_fake_node` + `robot_state_publisher` 是本地验证 TurtleBot3 TF 的便捷组合
- 养成保存 RViz 配置的习惯，避免每次手动添加 Display

命名约定提示： TurtleBot3 官方 URDF 使用 `wheel_left_link` / `wheel_right_link` 命名。在后续 3.2~3.5 中，我们的自定义机器人将使用 `left_wheel_link` / `right_wheel_link` 命名。两种风格都是合法的，关键是在同一项目中保持一致。

讲次 5：TF 中的关键坐标系：map / odom / base_link / sensor frames

知识点

移动机器人的四层坐标系

移动机器人导航与感知中，有四类"约定俗成"的核心坐标系：



map 坐标系

属性	说明
含义	全局地图坐标系，固定于环境
原点	通常对应地图左下角或建图起始位置
发布者	amcl、slam_toolbox、cartographer 等定位/建图节点
特点	全局一致，但有跳变（定位修正时）
与 odom 关系	由定位节点发布 map → odom 变换

odom 坐标系

属性	说明
含义	里程计坐标系，反映机器人相对启动点的位移
原点	机器人启动位置
发布者	底盘驱动节点、robot_localization
特点	局部精确、短期连续，长期有漂移
与 base_link 关系	发布 odom → base_link 变换

base_link 坐标系

属性	说明
含义	机器人本体中心坐标系
原点	通常设在底盘几何中心或旋转中心
定义者	URDF 中 <link name="base_link">
轴向	X 前、Y 左、Z 上（右手系）
与传感器关系	所有 sensor frame 的父级或祖先

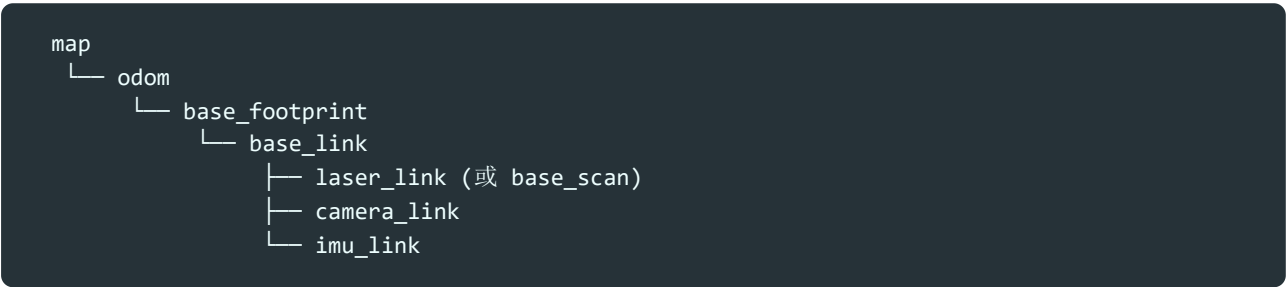
base_footprint

base_footprint 是 base_link 在地面 (Z=0) 的投影：



2D 导航 (Nav2) 通常使用 base_footprint 作为规划参考，避免高度干扰。

标准 TF 树 (导航场景)



关键约束： `map → odom` 变换由定位节点发布，`odom → base_link` 由里程计发布。二者职责分离，使得定位修正时只需调整 `map → odom`，不影响里程计的连续性。

为什么 map 和 odom 必须分离？

时间线：

t=0

机器人启动，odom 原点建立

t=10

里程计积累误差，odom 漂移 0.5m

t=11

AMCL 定位修正，发现实际偏差 0.5m

处理方式：

- 调整 map → odom 变换（补偿 0.5m 偏差）

- odom → base_link 保持连续（控制器不感知跳变）

如果只有一个全局坐标系，定位修正会导致位姿**跳变**，路径跟踪控制器会产生突变指令。

REP-105 标准

ROS 官方推荐标准 [REP-105](#) 定义了上述坐标系约定：

坐标系	方向约定	用途
map	X 东 / Y 北 / Z 上（或建图时任意）	全局路径规划
odom	与启动朝向一致	局部控制、短期融合
base_link	X 前 / Y 左 / Z 上	机器人本体参考
base_footprint	base_link 的地面投影	2D 导航

传感器坐标系

传感器	典型 frame 名	安装描述
2D 激光雷达	laser_link / base_scan	URDF fixed joint, 含 xyz + rpy
深度相机	camera_link / camera_depth_frame	有时含 optical frame
IMU	imu_link	通常接近质心
GPS	gps_link	天线位置

相机特殊坐标系：

camera_link → 相机物理安装系（ROS 标准：X 前 Y 左 Z 上）

camera_optical_frame → 光学系（Z 前 X 右 Y 下，符合 OpenCV 惯例）

两者之间通常有一个固定旋转：

```
<joint name="camera_optical_joint" type="fixed">
  <parent link="camera_link"/>
  <child link="camera_optical_frame"/>
  <origin xyz="0 0 0" rpy="-1.5708 0 -1.5708"/>
</joint>
```

命令行观察导航坐标系

启动 Nav2 仿真（预览，后续章节深入）：

```
# 查看当前所有 frame
ros2 run tf2_ros tf2_monitor

# 查看 map → base_link 的变换频率和延迟
ros2 run tf2_ros tf2_monitor map base_link
```

关键点

- 移动机器人标准四层坐标系：map（全局）→ odom（局部连续）→ base_link（本体）→ sensor frames
 - map 与 odom 分离是 REP-105 核心设计：定位修正只调整 map→odom，保证控制连续性
 - `base_footprint` 是 `base_link` 的地面投影，2D 导航的标准参考
 - 相机有 `camera_link`（物理系）和 `camera_optical_frame`（光学系）两套坐标
 - 所有坐标系应通过 URDF + 驱动节点正确发布到 TF 树
-

讲次 6：为什么需要机器人定位约定

知识点

没有约定时的混乱

假设两个开发者独立编写导航和感知模块：

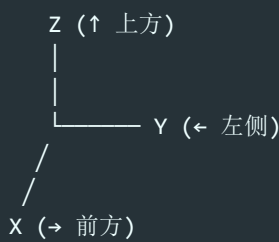
开发者 A 的坐标约定：	开发者 B 的坐标约定：
X 轴 = 前方	X 轴 = 右侧
Y 轴 = 左侧	Y 轴 = 前方
Z 轴 = 上方	Z 轴 = 上方
角度正方向 = 逆时针	角度正方向 = 顺时针

当 A 的导航节点说"向右转 90°"，B 的感知节点可能理解为"向前转 90°"。**没有统一约定，模块根本无法协作。**

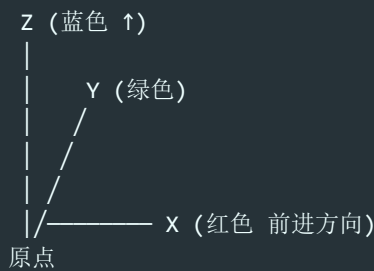
机器人学中的标准约定

1. 右手坐标系 (Right-Hand Rule)

ROS 采用右手坐标系：



验证方法：右手四指从 X 转向 Y，大拇指指向 Z 正方向。



ROS 标准 (REP-103/REP-105)
X: 前进方向 (Forward)
Y: 左方 (Left)
Z: 上方 (Up)
旋转: 右手定则
Roll: 绕X Pitch: 绕Y Yaw: 绕Z

2. 位姿表示约定

量	符号	单位	正方向
位置 x	x	米 (m)	前方为正
位置 y	y	米 (m)	左侧为正
位置 z	z	米 (m)	上方为正
滚转角 roll	φ	弧度 (rad)	绕 X 轴, 右手定则
俯仰角 pitch	θ	弧度 (rad)	绕 Y 轴, 右手定则
偏航角 yaw	ψ	弧度 (rad)	绕 Z 轴, 右手定则

3. 欧拉角顺序: RPY (Roll-Pitch-Yaw)

ROS 中 `<origin rpy="roll pitch yaw">` 表示按 **固定轴 (外旋, extrinsic)** 顺序的旋转:

```
R_total = R_z(yaw) × R_y(pitch) × R_x(roll)
```

即先绕固定 X 轴转 roll, 再绕固定 Y 轴转 pitch, 最后绕固定 Z 轴转 yaw (等价于内旋 Z-Y'-X'' 顺序)。

4. 四元数约定

ROS 使用 Hamilton 四元数约定:

```
q = (x, y, z, w)
其中 w 是标量部分 (scalar part)
```

```
# 验证: 绕 Z 轴旋转 90° 的四元数
# roll=0, pitch=0, yaw=π/2
# q = (0, 0, sin(π/4), cos(π/4)) = (0, 0, 0.7071, 0.7071)
ros2 run tf2_ros static_transform_publisher \
  --x 0 --y 0 --z 0 \
  --roll 0 --pitch 0 --yaw 1.5708 \
  --frame-id base_link --child-frame-id demo_yaw_90

# 另开终端查询
ros2 run tf2_ros tf2_echo base_link demo_yaw_90
```

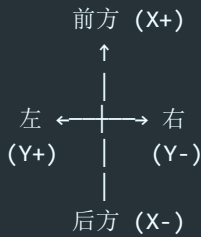
REP 标准文档体系

ROS 通过 REP (ROS Enhancement Proposal) 固化约定:

REP 编号	内容
REP-103	坐标轴、单位、旋转表示与四元数顺序等约定
REP-105	移动机器人坐标系层级 (map/odom/base_link)

移动机器人朝向约定

俯视图（移动机器人）：



正 yaw（偏航角）= 逆时针旋转（从上往下看）

这与数学中标准右手系一致：绕 Z 轴正旋转 = 逆时针。

坐标约定对 TF 查询的影响

```
# Python 示例：查询 base_link 在 map 中的位姿
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener

class PoseChecker(Node):
    def __init__(self):
        super().__init__('pose_checker')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

    def get_pose(self):
        try:
            transform = self.tf_buffer.lookup_transform(
                'map',          # 目标坐标系
                'base_link',    # 源坐标系
                rclpy.time.Time() # 最新时刻
            )
            t = transform.transform.translation
            r = transform.transform.rotation
            self.get_logger().info(
                f'Position: x={t.x:.2f}, y={t.y:.2f}, z={t.z:.2f}'
            )
            self.get_logger().info(
                f'Orientation (quat): x={r.x:.3f}, y={r.y:.3f}, '
                f'z={r.z:.3f}, w={r.w:.3f}'
            )
        except Exception as e:
            self.get_logger().warn(f'TF lookup failed: {e}')
```

理解约定后，才能正确解读上述输出：x 是前方距离，y 是左侧距离。

不同领域的坐标差异

领域	X 方向	Y 方向	注意
ROS 机器人	前方	左侧	右手系, RPY 欧拉角
计算机图形学	右	上	不同轴向
航空 (NED)	北	东	Z 向下
相机光学 (OpenCV)	右	下	Z 朝前 (光轴)

集成第三方库时，**必须显式处理坐标系转换**，不能假设约定相同。

实践：验证你的机器人是否遵循约定

```
# 1. 启动机器人模型
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py

# 2. 在 RViz 2 中检查 base_link 坐标轴
#   红色 X 轴应指向机器人前方
#   绿色 Y 轴应指向机器人左侧
#   蓝色 Z 轴应指向上方

# 3. 如果方向不对，需要修改 URDF 中 joint 的 rpy 值
```

关键点

- 统一坐标约定是模块协作的前提，否则"前方" "右转"等概念无法对齐
- ROS 采用右手坐标系：X 前、Y 左、Z 上；RPY 按 roll→pitch→yaw 顺序旋转
- REP-103/105 是移动机器人坐标约定的权威参考
- 四元数使用 Hamilton 约定 (x, y, z, w), w 为标量部分
- 集成第三方库 (OpenCV、PCL 等) 时必须注意坐标系差异并显式转换



讲次 7：移动机器人位姿表示

知识点

什么是位姿 (Pose)

位姿描述一个物体在空间中"在哪里"以及"朝向哪里", 由**位置 (Position)** 和**姿态 (Orientation)** 两部分组成。

$$\begin{aligned}\text{位姿} &= \text{位置} + \text{姿态} \\ &= (x, y, z) + (\text{旋转})\end{aligned}$$

在 ROS 中, 位姿用 `geometry_msgs/msg/Pose` 消息表示:

```
ros2 interface show geometry_msgs/msg/Pose
```

```
Point position
  float64 x
  float64 y
  float64 z
Quaternion orientation
  float64 x
  float64 y
  float64 z
  float64 w
```

带时间戳和坐标系引用的版本:

```
ros2 interface show geometry_msgs/msg/PoseStamped
```

```
std_msgs/Header header    # stamp + frame_id
Pose pose
```

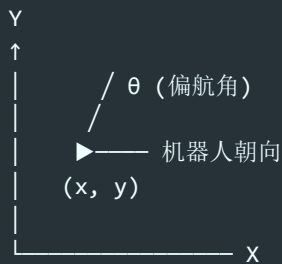
2D 位姿 vs 3D 位姿

移动机器人在平坦地面上运动时, 常简化为 **2D 位姿**:

```
2D 位姿: (x, y, θ)
- x, y: 平面位置 (米)
- θ: 绕 Z 轴的偏航角 yaw (弧度)

3D 位姿: (x, y, z, roll, pitch, yaw)
或 (x, y, z, qx, qy, qz, qw)
```

场景	使用维度	原因
室内平地导航	2D (x, y, θ)	高度和俯仰变化可忽略
无人机	3D 全位姿	六自由度运动
机械臂末端	3D 全位姿	空间操作
坡道行驶	3D (含 pitch)	需要俯仰信息



2D 位姿 = (x, y, θ)
3D 位姿 = (x, y, z, roll, pitch, yaw)
= (translation, quaternion)

2D 位姿的数学表示

平面上的位姿可用向量表示：

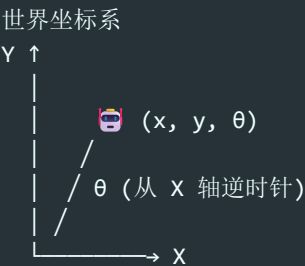
$$\xi = [x, y, \theta]^T$$

其中：

(x, y) = 位置向量

θ = 偏航角（从 x 轴逆时针测量）

图示：



位姿的三种等价表示

表示法	形式	优点	缺点
欧拉角 (RPY)	(roll, pitch, yaw)	直观易懂	万向节死锁
旋转矩阵	3×3 矩阵	无歧义、易组合	9 个参数，有约束
四元数	(x, y, z, w)	紧凑、无奇点、插值平滑	不直观

ROS 消息中姿态统一使用四元数，但 URDF 和调试中常用 RPY。

四元数基础

四元数 $q = (x, y, z, w)$ 表示旋转，满足：

$|q| = \sqrt{x^2 + y^2 + z^2 + w^2} = 1$ （单位四元数）

特殊值：

旋转	四元数 (x, y, z, w)
无旋转（恒等）	(0, 0, 0, 1)
绕 Z 轴 90°	(0, 0, 0.7071, 0.7071)
绕 Z 轴 180°	(0, 0, 1, 0)
绕 X 轴 180°	(1, 0, 0, 0)

RPY 与四元数互转

```
# Python 示例: RPY ↔ 四元数
import math
from geometry_msgs.msg import Quaternion

def euler_to_quaternion(roll, pitch, yaw):
    """RPY (弧度) 转四元数"""
    cy = math.cos(yaw * 0.5)
    sy = math.sin(yaw * 0.5)
    cp = math.cos(pitch * 0.5)
    sp = math.sin(pitch * 0.5)
    cr = math.cos(roll * 0.5)
    sr = math.sin(roll * 0.5)

    q = Quaternion()
    q.w = cr * cp * cy + sr * sp * sy
    q.x = sr * cp * cy - cr * sp * sy
    q.y = cr * sp * cy + sr * cp * sy
    q.z = cr * cp * sy - sr * sp * cy
    return q

# 示例: yaw = 90°
q = euler_to_quaternion(0, 0, math.pi / 2)
print(f'x={q.x:.4f}, y={q.y:.4f}, z={q.z:.4f}, w={q.w:.4f}')
# 输出: x=0.0000, y=0.0000, z=0.7071, w=0.7071
```

C++ 版本 (使用 tf2) :

```
#include <tf2/LinearMath/Quaternion.h>
#include <tf2_geometry_msgs/tf2_geometry_msgs.hpp>

// RPY 转四元数
tf2::Quaternion q;
q.setRPY(0.0, 0.0, M_PI / 2.0); // roll, pitch, yaw
// q = (0, 0, 0.7071, 0.7071)
```

位姿的组合

若已知 A 在 B 中的位姿 ${}^B T_A$, B 在 C 中的位姿 ${}^C T_B$, 则 A 在 C 中的位姿:

$${}^C T_A = {}^C T_B \times {}^B T_A$$

这是 TF 系统的核心运算——沿变换链连乘。

```
map —(map→odom)→ odom —(odom→base)→ base_link
   T_map_odom      T_odom_base

T_map_base = T_map_odom × T_odom_base
```

发布位姿到 ROS 2

```
# Python: 发布 PoseStamped (完整可运行示例)
import math
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseStamped, Quaternion

def euler_to_quaternion(roll, pitch, yaw):
    """RPY (弧度) 转四元数"""
    cy = math.cos(yaw * 0.5)
    sy = math.sin(yaw * 0.5)
    cp = math.cos(pitch * 0.5)
    sp = math.sin(pitch * 0.5)
    cr = math.cos(roll * 0.5)
    sr = math.sin(roll * 0.5)

    q = Quaternion()
    q.w = cr * cp * cy + sr * sp * sy
    q.x = sr * cp * cy - cr * sp * sy
    q.y = cr * sp * cy + sr * cp * sy
    q.z = cr * cp * sy - sr * sp * cy
    return q

class PosePublisher(Node):
    def __init__(self):
        super().__init__('pose_publisher')
        self.pub = self.create_publisher(PoseStamped, '/robot_pose', 10)
        self.timer = self.create_timer(1.0, self.publish_pose)

    def publish_pose(self):
        msg = PoseStamped()
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'map'
        msg.pose.position.x = 1.0
        msg.pose.position.y = 2.0
        msg.pose.position.z = 0.0
        msg.pose.orientation = euler_to_quaternion(0, 0, math.pi / 4)
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = PosePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()
```



```
if __name__ == '__main__':  
    main()
```

```
# 命令行查看  
ros2 topic echo /robot_pose
```

位姿可视化

在 RViz 2 中:

```
Add → Pose → Topic: /robot_pose
```

位姿箭头显示位置（箭头尾部）和朝向（箭头方向）。

关键点

- 位姿 = 位置 (x,y,z) + 姿态（旋转），ROS 消息为 `geometry_msgs/Pose` / `PoseStamped`
 - 平地移动机器人常用 2D 位姿 (x, y, θ)，全空间运动需 3D 位姿
 - 姿态有三种等价表示：欧拉角（直观）、旋转矩阵（精确）、四元数（ROS 标准）
 - 位姿组合通过变换矩阵连乘： ${}^cT_a = {}^cT_\beta \times {}^BT_a$
 - TF 系统本质上是维护 and 查询位姿变换链
-

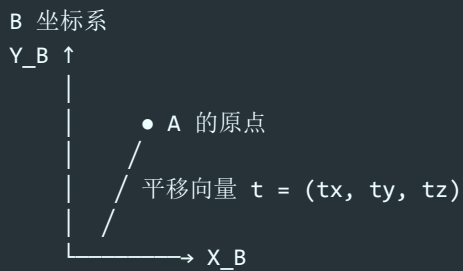
讲次 8: 平移向量 (Translation Vector)

知识点

平移的物理含义

平移 (Translation) 描述一个坐标系原点在另一个坐标系中的位置偏移。

坐标系 A 的原点在 B 中的位置:



$t = (tx, ty, tz) = A$ 原点在 B 中的坐标

平移向量在 ROS 中的表示

```
ros2 interface show geometry_msgs/msg/Vector3
```

```
float64 x
float64 y
float64 z
```

在 `TransformStamped` 中:

```
geometry_msgs/Transform transform
  Vector3 translation    # 平移向量 (x, y, z)
  Quaternion rotation    # 旋转 (四元数)
```

点的平移变换

给定点 P 在坐标系 A 中的坐标 $^A p$, 求其在坐标系 B 中的坐标 $^B p$:

仅有平移 (无旋转) 时:

$$^B p = ^A p + t$$

其中 t 是 A 原点在 B 中的位置向量

示例:

A 原点在 B 中位于 (1, 2, 0)
点 P 在 A 中坐标为 (3, 1, 0)

P 在 B 中坐标 = (3+1, 1+2, 0+0) = (4, 3, 0)

平移的矩阵表示

平移可以用齐次坐标写成矩阵形式（为后续讲次铺垫）：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

平移不是向量相加那么简单

当涉及**旋转**时，平移和旋转不可分离：

错误： ${}^B p = {}^A p + t$ （仅无旋转时成立）
正确： ${}^B p = R \times {}^A p + t$ （R 为旋转矩阵）

这就是为什么需要齐次变换矩阵将旋转和平移统一表达。

URDF 中的平移

URDF 的 `<origin>` 标签用 `xyz` 属性定义平移：

```
<joint name="laser_joint" type="fixed">
  <parent link="base_link"/>
  <child link="laser_link"/>
  <origin xyz="0.1 0.0 0.2" rpy="0 0 0"/>
  <!--
    含义: laser_link 原点在 base_link 中位于
    x=0.1m（前方 10cm）
    y=0.0m（无左右偏移）
    z=0.2m（上方 20cm）
  -->
</joint>
```

动手计算

练习：TurtleBot3 Burger 的激光雷达安装位置

```
# 查看 URDF 中激光 joint 的 origin
grep -A5 'scan_joint\|laser' \
/opt/ros/jazzy/share/turtlebot3_gazebo/urdf/turtlebot3_burger.urdf
```

Jazzy 中 TurtleBot3 Burger 的 `scan_joint` 实测为 `xyz="-0.032 0 0.172"` , 含义:

分量	值	物理含义
x	-0.032	激光雷达原点相对 base_link 向后 3.2cm
y	0	无左右偏移
z	0.172	激光雷达在 base_link 上方 17.2cm

验证:

```
ros2 run tf2_ros tf2_echo base_link base_scan
# 检查 Translation 是否接近 [-0.032, 0, 0.172]
```

平移的逆变换

若已知 A 系在 B 系中的位姿为 (R, t) (R 为姿态、t 为平移) , 则反过来 B 系在 A 系中的平移:

$$t_{B_in_A} = -R^{-1} \times t = -R^T \times t \quad (R \text{ 为正交矩阵时 } R^{-1} = R^T)$$

两坐标系之间无相对旋转 ($R = I$) 时简化为:

$$t_{B_in_A} = -t$$

Python 实践: 平移变换

```
import numpy as np

def translate_point(point, translation):
    """
    将点从坐标系 A 变换到坐标系 B (仅平移)
    point: 点在 A 中的坐标 [x, y, z]
    translation: A 原点在 B 中的位置 [tx, ty, tz]
    """
    return np.array(point) + np.array(translation)

# 激光雷达点云中的点
point_in_laser = [2.0, 0.5, 0.0] # 激光系下 2m 处障碍物
laser_in_base = [-0.032, 0.0, 0.172] # TurtleBot3 Burger 中 base_scan 的位置

point_in_base = translate_point(point_in_laser, laser_in_base)
print(f'障碍物在 base_link 中: {point_in_base}')
# 输出: [1.968, 0.5, 0.172]
```

平移在 TF 查询中的体现

```
# tf2_echo 输出分解
ros2 run tf2_ros tf2_echo base_link base_scan

# 输出:
# - Translation: [-0.032, 0.000, 0.172]
#   → 激光雷达在 base_link 后方 3.2cm、上方 17.2cm
# - Rotation: in RPY (radian) [0.000, -0.000, 0.000]
#   → base_scan 与 base_link 朝向一致
```

关键点

- 平移向量 $t = (t_x, t_y, t_z)$ 描述子坐标系原点在父坐标系中的位置
 - 无旋转时, 点的变换为 ${}^Bp = {}^Ap + t$
 - URDF 中 `<origin xyz="...">` 定义关节的平移分量
 - 有旋转时平移不能独立处理, 必须与旋转矩阵联合运算
 - `tf2_echo` 的 Translation 行直接给出平移向量, 可与 URDF 对照验证
-

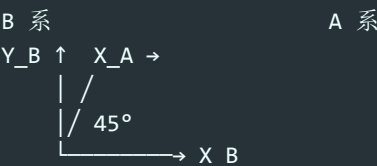
讲次 9：旋转矩阵 (Rotation Matrix)

知识点

为什么需要旋转矩阵

平移向量只能表达"原点偏移", 无法描述"朝向不同"。当两个坐标系之间存在**角度偏差**时, 需要旋转矩阵。

坐标系 A 的 X 轴指向 B 的 Y 轴方向:



旋转矩阵的定义

3×3 旋转矩阵 R 将坐标系 A 中的向量变换到坐标系 B:

$${}^B\mathbf{v} = \mathbf{R} \times {}^A\mathbf{v}$$

其中 R 是正交矩阵: $\mathbf{R}^T \mathbf{R} = \mathbf{I}$, $\det(\mathbf{R}) = +1$

旋转矩阵的**列向量**是 A 的各轴在 B 中的方向:

$$\mathbf{R} = [\text{ } x_A_in_B \mid y_A_in_B \mid z_A_in_B \text{ }]$$

$$\begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \quad \begin{array}{l} \text{第 1 列} = \text{A 的 X 轴在 B 中的方向} \\ \text{第 2 列} = \text{A 的 Y 轴在 B 中的方向} \\ \text{第 3 列} = \text{A 的 Z 轴在 B 中的方向} \end{array}$$

基本旋转矩阵

绕单轴旋转 θ 角的三个基本旋转矩阵:

绕 X 轴旋转 (Roll) :

$$\mathbf{R}_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) \\ 0 & \sin(\theta) & \cos(\theta) \end{bmatrix}$$

绕 Y 轴旋转 (Pitch) :

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) \\ 0 & 1 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) \end{bmatrix}$$

绕 Z 轴旋转 (Yaw) :

$$R_z(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

RPY 组合旋转

ROS 中 `<origin rpy="roll pitch yaw">` 对应的旋转矩阵:

$$R = R_z(\text{yaw}) \times R_y(\text{pitch}) \times R_x(\text{roll})$$

注意顺序: 这是**内在旋转** (相对于当前坐标系), 与外在旋转顺序相反。

计算示例

例: $\text{rpy} = (0, 0, \pi/2)$, 即绕 Z 轴旋转 90°

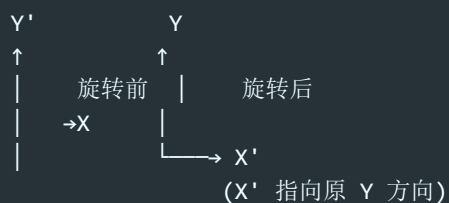
$$R = R_z(\pi/2) = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

验证: A 的 X 轴 $[1, 0, 0]$ 变换后在 B 中:

$${}^B X = R \times [1, 0, 0]^T = [0, 1, 0]^T$$

即 A 的 X 轴指向 B 的 Y 轴方向——符合 90° 逆时针旋转。

绕 Z 轴旋转 $\theta = 90^\circ$:



$$R_z(90^\circ) \cdot [1, 0, 0]^T = [0, 1, 0]^T$$

Python 实现

```
import numpy as np

def rotation_matrix_rpy(roll, pitch, yaw):
    """根据 RPY 角（弧度）计算旋转矩阵"""
    Rx = np.array([
        [1, 0, 0],
        [0, np.cos(roll), -np.sin(roll)],
        [0, np.sin(roll), np.cos(roll)]
    ])
    Ry = np.array([
        [np.cos(pitch), 0, np.sin(pitch)],
        [0, 1, 0],
        [-np.sin(pitch), 0, np.cos(pitch)]
    ])
    Rz = np.array([
        [np.cos(yaw), -np.sin(yaw), 0],
        [np.sin(yaw), np.cos(yaw), 0],
        [0, 0, 1]
    ])
    return Rz @ Ry @ Rx # 矩阵乘法, 顺序: 先 Rx, 再 Ry, 最后 Rz

# 绕 Z 轴 90°
R = rotation_matrix_rpy(0, 0, np.pi / 2)
print(R)
# [[0. -1.  0.]
#  [1.  0.  0.]
#  [0.  0.  1.]]

# 变换一个点
point_a = np.array([1.0, 0.0, 0.0])
point_b = R @ point_a
print(f'变换后: {point_b}') # [0. 1. 0.]
```

C++ 实现 (tf2 库)

```
#include <tf2/LinearMath/Matrix3x3.h>
#include <tf2/LinearMath/Quaternion.h>

// 从四元数获取旋转矩阵
tf2::Quaternion q(0, 0, 0.7071, 0.7071); // yaw=90°
tf2::Matrix3x3 rot_mat(q);

double r11, r12, r13, r21, r22, r23, r31, r32, r33;
rot_mat.getValue(r11, r12, r13, r21, r22, r23, r31, r32, r33);
```


旋转矩阵的性质

性质	公式	含义
正交性	$R^T R = I$	保持向量长度和角度
行列式	$\det(R) = +1$	保持手性（不镜像）
逆矩阵	$R^{-1} = R^T$	逆旋转 = 转置
组合	$R_3 = R_2 \times R_1$	连续旋转可连乘

旋转矩阵 vs 四元数

维度	旋转矩阵	四元数
参数数量	9 (3 独立)	4 (3 独立)
奇点	无	无
插值	不直接支持	SLERP 平滑插值
组合	矩阵乘法	四元数乘法
ROS 消息	不直接使用	<code>geometry_msgs/Quaternion</code>

TF 内部使用四元数存储旋转，但数学推导中旋转矩阵更直观。

从 tf2_echo 解读旋转

```
ros2 run tf2_ros static_transform_publisher \
  --x 0 --y 0 --z 0 \
  --roll 0 --pitch 0 --yaw 1.5708 \
  --frame-id base_link --child-frame-id demo_yaw_90

# 另开终端
ros2 run tf2_ros tf2_echo base_link demo_yaw_90
```

输出：

```
- Rotation: in Quaternion [0.000, 0.000, 0.707, 0.707]
           in RPY (radian) [0.000, -0.000, 1.571]
           in RPY (degree) [0.000, -0.000, 90.000]
```

解读： `demo_yaw_90` 相对于 `base_link` 只有 yaw 约 90°，无 roll 和 pitch 偏差。

关键点

- 旋转矩阵 R 是 3×3 正交矩阵，将向量从子坐标系变换到父坐标系： ${}^B\mathbf{v} = \mathbf{R} \times {}^A\mathbf{v}$
- 列向量是子坐标系各轴在父坐标系中的方向

- ROS RPY 约定: $R = R_z(\text{yaw}) \times R_y(\text{pitch}) \times R_x(\text{roll})$
 - 旋转矩阵逆等于转置: $R^{-1} = R^T$
 - TF 内部用四元数存储旋转, 但旋转矩阵是理解变换数学的基础
-

讲次 10: 齐次变换矩阵 (Transformation Matrix)

知识点

统一旋转与平移

单独使用旋转矩阵和平移向量时，点的变换公式为：

$${}^B p = R \times {}^A p + t$$

这种形式**不能通过矩阵连乘**来表达变换链。齐次变换矩阵将旋转和平移统一为 4×4 矩阵：

$${}^B p = T \times {}^A p \quad (\text{齐次坐标下})$$

齐次变换矩阵的结构

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

左上角 3×3 = 旋转矩阵 R

右上角 3×1 = 平移向量 t

底行 = $[0, 0, 0, 1]$

齐次坐标

普通 3D 点 (x, y, z) 扩展为 4D 齐次坐标 $(x, y, z, 1)$ ：

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

变换链的矩阵连乘

这是齐次变换矩阵的核心优势：

$A \rightarrow B \rightarrow C$ 的变换链:

$${}^C T_A = {}^C T_B \times {}^B T_A$$

即: 从 A 到 C 的变换 = 从 B 到 C $\times$ 从 A 到 B

变换链: map $\rightarrow$ odom $\rightarrow$ base_link $\rightarrow$ base_scan

$$T_{\text{map_scan}} = T_{\text{map_odom}} \times T_{\text{odom_base}} \times T_{\text{base_scan}}$$



图示:

base_scan $\xrightarrow{(T_{\text{base_scan}})}$ base_link $\xrightarrow{(T_{\text{odom_base}})}$ odom

$$T_{\text{odom_scan}} = T_{\text{odom_base}} \times T_{\text{base_scan}}$$

注意乘法顺序: 右乘先作用 (从右向左读变换链)。

构建齐次变换矩阵

```
import numpy as np

def make_transform(translation, rpy):
    """
    构建 4x4 齐次变换矩阵
    translation: [tx, ty, tz]
    rpy: [roll, pitch, yaw] 弧度
    """
    roll, pitch, yaw = rpy
    Rx = np.array([[1, 0, 0],
                   [0, np.cos(roll), -np.sin(roll)],
                   [0, np.sin(roll), np.cos(roll)]])
    Ry = np.array([[np.cos(pitch), 0, np.sin(pitch)],
                   [0, 1, 0],
                   [-np.sin(pitch), 0, np.cos(pitch)]])
    Rz = np.array([[np.cos(yaw), -np.sin(yaw), 0],
                   [np.sin(yaw), np.cos(yaw), 0],
                   [0, 0, 1]])
    R = Rz @ Ry @ Rx

    T = np.eye(4)
    T[:3, :3] = R
    T[:3, 3] = translation
    return T

# TurtleBot3 Burger 的 base_scan 变换
T_base_scan = make_transform([-0.032, 0, 0.172], [0, 0, 0])
print(T_base_scan)
```

输出:

```
[[ 1.    0.    0.   -0.032]
 [ 0.    1.    0.    0.   ]
 [ 0.    0.    1.    0.172]
 [ 0.    0.    0.    1.   ]]
```

逆变换

齐次变换矩阵的逆:

$$T^{-1} = \begin{bmatrix} R^T & -R^T t \\ 0 & 1 \end{bmatrix}$$

```
def invert_transform(T):
    """求齐次变换矩阵的逆"""
    R = T[:3, :3]
    t = T[:3, 3]
    T_inv = np.eye(4)
    T_inv[:3, :3] = R.T
    T_inv[:3, 3] = -R.T @ t
    return T_inv

# 验证: T × T-1 = I
T_inv = invert_transform(T_base_scan)
print(np.allclose(T_base_scan @ T_inv, np.eye(4))) # True
```

完整示例: 激光点变换到 base_link

```
# 激光雷达检测到的障碍物
point_laser = np.array([2.0, 0.5, 0.0, 1.0]) # 齐次坐标

# base_scan 到 base_link 的变换
T_base_scan = make_transform([-0.032, 0, 0.172], [0, 0, 0])

# 变换到 base_link
point_base = T_base_scan @ point_laser
print(f'障碍物在 base_link: ({point_base[0]:.2f}, {point_base[1]:.2f}, {point_base[2]:.2f})')
# 输出: (1.97, 0.50, 0.17)
```

TF 与齐次变换矩阵的对应

TF 概念	矩阵概念
TransformStamped	4×4 齐次变换矩阵
translation (x,y,z)	矩阵右上角 t
rotation (四元数)	矩阵左上角 R
TF 变换链查询	矩阵连乘
tf2_echo A B	输出 B 在 A 中的 T

使用 tf2 进行矩阵运算

```
import rclpy
from rclpy.node import Node
from tf2_ros import Buffer, TransformListener
import tf2_geometry_msgs # noqa: F401 - 注册 PointStamped 等类型的转换支持
from geometry_msgs.msg import PointStamped

class TransformDemo(Node):
    def __init__(self):
        super().__init__('transform_demo')
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

    def transform_point(self):
        point_in_laser = PointStamped()
        point_in_laser.header.frame_id = 'base_scan'
        point_in_laser.point.x = 2.0
        point_in_laser.point.y = 0.5
        point_in_laser.point.z = 0.0

        try:
            point_in_base = self.tf_buffer.transform(
                point_in_laser, 'base_link', timeout=rclpy.duration.Duration(seconds=1.0)
            )
            p = point_in_base.point
            self.get_logger().info(
                f'Laser point (2.0, 0.5, 0) → base_link: ({p.x:.2f}, {p.y:.2f}, {p.z:.2f})'
            )
        except Exception as e:
            self.get_logger().error(f'Transform failed: {e}')
```

关键点

- 齐次变换矩阵 T 是 4×4 矩阵，左上角 R（旋转）+ 右上角 t（平移），统一表达刚体变换
- 变换链通过矩阵连乘实现： ${}^cT_a = {}^cT_b \times {}^bT_a$ ，乘法从右向左读
- 逆变换：T⁻¹ 的旋转部分为 R^T，平移部分为 -R^Tt
- TF 系统的底层数学就是维护和查询齐次变换矩阵链

- `tf_buffer.transform()` 自动完成点的坐标系变换，无需手动矩阵运算
-

讲次 11：坐标系间的变换计算

知识点

本讲综合目标

综合运用前序讲次的数学工具，完成以下典型计算：1. 已知变换链，求任意两坐标系间的变换 2. 已知点的坐标，变换到目标坐标系 3. 验证 URDF 与 TF 输出的一致性 4. 使用 ROS 2 工具手动完成变换查询

问题 1：沿变换链求复合变换

给定 TF 树：

```
map → odom → base_link → base_scan
```

已知：- T_{map_odom} : map 到 odom 的变换 - T_{odom_base} : odom 到 base_link 的变换 - T_{base_scan} : base_link 到 base_scan 的变换

求： T_{map_scan} (base_scan 在 map 中的位姿)

```
 $T_{map\_scan} = T_{map\_odom} \times T_{odom\_base} \times T_{base\_scan}$ 
```

数值示例：

```
import numpy as np

# 假设已知变换（简化数值）
T_map_odom = make_transform([1.0, 2.0, 0.0], [0, 0, 0.0])      # 机器人在 map 中 (1,2)，朝向 0°
T_odom_base = make_transform([0.0, 0.0, 0.0], [0, 0, 0.0])    # odom 与 base 重合
T_base_scan = make_transform([-0.032, 0.0, 0.172], [0, 0, 0.0]) # Burger 激光雷达安装位姿

T_map_scan = T_map_odom @ T_odom_base @ T_base_scan
print("base_scan 在 map 中的位置:", T_map_scan[:3, 3])
# 输出: [0.968 2.    0.172]
```

问题 2：点的坐标变换

激光雷达检测到障碍物在 base_scan 中坐标为 (3.0, 1.0, 0.0)，求其在 map 中的坐标：

```
point_laser = np.array([3.0, 1.0, 0.0, 1.0])
point_map = T_map_scan @ point_laser
print(f'障碍物在 map 中: ({point_map[0]:.2f}, {point_map[1]:.2f}, {point_map[2]:.2f})')
# 输出: (3.97, 3.00, 0.17)
```


问题 3: 逆变换——已知子系中的点, 求父系坐标

若已知 T_{parent_child} , 求 child 系中点 p 在 parent 系中的坐标:

```
p_parent = T_parent_child × p_child
```

若已知 T_{parent_child} , 求 parent 系中点 p 在 child 系中的坐标:

```
p_child = T_parent_child-1 × p_parent
```

```
# 将 map 中的目标点 (5, 3, 0) 转换到 base_scan 系
point_map = np.array([5.0, 3.0, 0.0, 1.0])
T_laser_map = invert_transform(T_map_scan)
point_laser = T_laser_map @ point_map
print(f'目标在 base_scan 中: ({point_laser[0]:.2f}, {point_laser[1]:.2f}, {point_laser[2]:.2f})')
```

问题 4: 时间插值

TF 是**时间戳化**的变换。当 Buffer 中已有查询时刻前后的样本时, TF2 可以在两者之间插值:

```
t=1.0: T1 = (x=1.0, y=0.0)
t=2.0: T2 = (x=3.0, y=0.0)

查询 t=1.5 时:
T1.5 = interpolate(T1, T2, 0.5)
      → (x=2.0, y=0.0)    (平移线性插值, 旋转 SLERP)
```

TF2 不会预测尚未收到的未来变换。查询时间晚于 Buffer 中最新样本, 或早于缓存最旧样本时, 通常会得到 extrapolation 错误; `timeout` 只表示最多等待数据到达, 不是允许无限外推。

```
# 查询特定时刻的变换
transform = tf_buffer.lookup_transform(
    'map',                # 目标坐标系
    'base_link',          # 源坐标系
    rclpy.time.Time(seconds=1, nanoseconds=500000000), # 指定时刻
    timeout=rclpy.duration.Duration(seconds=1.0)
)
```

使用 ROS 2 命令行工具验证

方法 1: tf2_echo (两个坐标系间)

```
source /opt/ros/jazzy/setup.bash
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py &
sleep 2

# 查询变换
ros2 run tf2_ros tf2_echo base_link base_scan
```

方法 2: tf2_monitor (频率和延迟)

```
ros2 run tf2_ros tf2_monitor
```

方法 3: 手动发布变换并验证

```
# 终端 1: 发布静态变换 map → odom (ROS 2 Jazzy 标志式 CLI)
ros2 run tf2_ros static_transform_publisher \
  --x 1.0 --y 2.0 --z 0.0 \
  --roll 0.0 --pitch 0.0 --yaw 0.0 \
  --frame-id map --child-frame-id odom

# 终端 2: 查询 map → base_scan (需要 robot_state_publisher 运行)
ros2 run tf2_ros tf2_echo map base_scan
```

`static_transform_publisher` 参数格式 (Jazzy) :

```
--x --y --z --roll --pitch --yaw --frame-id <parent> --child-frame-id <child>
# 或使用四元数:
--x --y --z --qx --qy --qz --qw --frame-id <parent> --child-frame-id <child>
```

综合练习

练习 1: 手动计算

```
已知:
T_base_laser: xyz=(0.1, 0, 0.2), rpy=(0, 0,  $\pi/2$ )
激光点 p_laser = (1.0, 0, 0)

求: p_base = ?
```

► 参考答案

练习 2: 编程验证

编写 Python 节点, 订阅激光点云, 将第一个点变换到 `base_link` 并打印。

练习 3: TF 树断裂诊断

```
# 故意不启动 robot_state_publisher, 观察错误
ros2 run tf2_ros tf2_echo map base_link
# 预期: Could not find a connection between 'map' and 'base_link'
```

变换计算速查表

操作	公式	ROS 2 工具
复合变换	${}^cT_a = {}^cT_b \times {}^bT_a$	<code>tf2_echo A C</code>
点变换	$p_B = {}^BT_a \times p_A$	<code>tf_buffer.transform()</code>
逆变换	${}^AT_b = ({}^BT_a)^{-1}$	<code>tf2_echo B A</code> (交换顺序)
静态变换发布	手动指定	<code>static_transform_publisher</code>
变换链可视化	—	<code>view_frames</code>

从数学到工程

数学层		TF 系统层		应用层
齐次变换矩阵	→	TF Buffer	→	点云坐标变换
矩阵连乘	→	变换链查询	→	导航路径规划
逆矩阵	→	反向查询	→	目标检测定位
插值	→	时间戳查询	→	传感器融合

关键点

- 变换链通过矩阵连乘计算： $T_{A_C} = T_{A_B} \times T_{B_C}$ ，从右向左读
- 点的变换： $p_{target} = T_{target_source} \times p_{source}$ （齐次坐标）
- 逆变换将点从父系变换到子系： $p_{child} = T^{-1} \times p_{parent}$
- `static_transform_publisher` 可手动发布固定变换，是调试 TF 的利器
- TF 查询支持时间戳插值，是处理传感器时间不同步的关键机制